

Vectortiles mit MapLibre – Workshop-Handout

Jakob Listabarth
27. Mai. 2026, 09:00 Uhr
74. DKK, Dresden

Lernziele

- Kontext von Webmapping und *MapLibre* verstehen
- *MapLibre Style Specification* verstehen und anwenden
- Praktische Anwendung von MapLibre in einer interaktiven Karte
 - Daten laden und als Kartenebene darstellen
 - Grundlegende Karten-Interaktionen implementieren

MapLibre und Vectortiles

Was ist MapLibre?

MapLibre ist ein Open-Source-Fork von *mapbox-gl-js*. Die JavaScript-Bibliothek verwendet die *WebGL*-Technologie (unter Verwendung der GPU - *Graphic Processing Unit*), um Karten im Browser darzustellen. Im Unterschied zu anderen Karten-Bibliotheken für das Web (wie z. B. *Leaflet* oder *OpenLayers*), die auf die Canvas 2D API (bzw. auf *SVGs*) setzen, ermöglicht *WebGL* eine deutlich bessere Performance. Der Performancevorteil ist besonders merkbar bei der Darstellung großer Datenmengen oder komplexer Kartenstile.

Was sind Vectortiles?

Kacheln (*tiles*) sind ein gängiges Format für die Bereitstellung von Geodaten im Web. Dabei wird die quadratische Karte pro Zoomstufe in jeweils 4 kleinere Kacheln unterteilt (*Quadtree*).

Vectortiles sind ein Datenformat für das Speichern von Geodaten. Diese Daten bestehen aus Geometrien (Punkten, Linien, Polygonen) und deren Attributen (nicht-räumliche, beschreibende Daten). Im Gegensatz zu Rastertiles (*Web Map Service*) beinhalten Vectortiles tatsächlich **nur die geometrischen Informationen** und **nicht das Aussehen**. Das Aussehen wird erst im Browser durch die Anwendung von Stilen (*styles*) definiert und gerendert. Das bedeutet, dass die Darstellung erst direkt beim Kartenaufruf erfolgt. Dadurch kann die Karte viel leichter an die individuellen Bedarfe der Kartennutzer*innen bzw. den jeweiligen Nutzungskontext angepasst werden.

Vorteile von Vectortiles

- dynamisch und anpassbar (in Real-Time)
 - Accessibility: Schriftgröße, Kontrast, Farben (basierend auf Nutzer*innenpräferenzen und Nutzungskontext)
 - Sprache (Internationalisierung)
 - Inhalte (Filter als Teil der Stil-Definition)
- kontinuierlicher Zoom (niemals verpixelt)
- geringere Speichergröße als Rastertiles (abhängig von Anzahl der Stile und Generalisierung)

Nachteile von Vectortiles

- Qualitätsverlust: weniger Details, Auflösung, Anti-Aliasing
- Begrenzt in Bezug auf visuelle Details oder Stile (z. B. keine Schatten, Verläufe etc.)
- benötigt relative leistungsstarke Geräte (WebGL)

MapLibre Style Specification

Wichtige Bestandteile der MapLibre Style Specification sind:

- **Sources:** Datenquellen, die für unterschiedliche Ebenen der Karte verwendet werden (z. B. Vectortiles, Rastertiles, GeoJSON, etc.)
- **Layers:** Definieren die Darstellung der Datenquellen (z. B. Punkte für Hauptstädte, Linien für Straßen, etc.)
- **Glyphs:** Schriften für die Darstellung von Texten auf der Karte
- **Sprites:** Grafische Symbole für die Darstellung von Punkten oder Linien auf der Karte

Sources (Datenquellen)

Datenquellen werden als Javascript *object* (s. u.) definiert, durch einen eindeutigen Schlüssel gekennzeichnet, und enthalten:

- die Art der Datenquelle (z. B. Vectortiles, Rastertiles, GeoJSON, etc.)
- die URL oder den Pfad zu der Datenquelle

Layer (Kartenebenen)

Alle Layers werden als ein *array* (s. u.) definiert, das die verschiedenen Ebenen der Karte beinhaltet. Jede Ebene enthält Informationen über

- die Datenquelle, die für diese Ebene verwendet wird
- die Art der Darstellung
- sowie die Stileigenschaften für die Darstellung (abhängig von der Art der Darstellung)

Wichtige Eigenschaften von Layern sind:

- **id:** Ein eindeutiger Bezeichner für die Ebene
- **type:** Die Art der Darstellung (z. B. **symbol** für Icons, **line** für Linien, **fill** für Polygone, etc.)
- **source:** Der Schlüssel der Datenquelle, die für diese Ebene verwendet wird
- **source-layer:** Der Name der Layer innerhalb der Datenquelle (nur bei Vectortiles)
- **paint:** Ein Objekt, das die Stileigenschaften für die Darstellung der Daten definiert (z. B. **line-color**, **line-width**, etc.)
- **layout:** Ein Objekt, das die Layout-Eigenschaften für die Darstellung der Daten definiert (z. B. **text-field** für die Anzeige von Texten, **icon-image**)

JavaScript-Basics

JavaScript ist eine Programmiersprache, die hauptsächlich für die Entwicklung von Webseiten und Webanwendungen, also im Browser, ausgeführt wird. Mittlerweile wird JavaScript aber auch außerhalb des Browsers (z. B. auf Servern) verwendet. JavaScript ermöglicht es, Webseiten interaktiv zu gestalten und dynamische Inhalte zu erstellen. An dieser Stelle werden nur die aller wichtigsten Grundlagen behandelt, um die praktische Anwendung von MapLibre zu verstehen.

Variablen und Datentypen

Ein wichtiges Konzept in JavaScript ist die Verwendung von Variablen. Variablen kann ein Wert zugewiesen werden, der später im Code verwendet oder verändert werden kann.

```
// Variablen werden mit dem Schlüsselwort `const` oder `let`  
deklariert. `const` für unveränderliche Werte, `let` für  
veränderliche Werte.  
const name = "Maplibre Workshop"; // der Variable `name` wird  
ein string (Zeichenkette) zugewiesen
```

Am Ende jeder Anweisung wird ein Semikolon (;) gesetzt. Die Verwendung des Semikolons ist in viele Fällen optional, wird aber empfohlen. Um Anmerkungen im Code zu machen, können

Kommentare verwendet werden. Einzeilige Kommentare beginnen mit `//`, mehrzeilige Kommentare werden von `/*` und `*/` eingeschlossen.

```
// Das ist ein einzeiliger Kommentar
/*
Das ist ein mehrzeiliger Kommentar,
der über 2 Zeilen geht.
*/
```

Es gibt unterschiedliche *einfache* Datentypen in JavaScript, wie z. B. Strings (Zeichenketten), Zahlen, Booleans (wahr/falsch).

```
const myString = "Hello, World!"; // String
const myNumber = 42; // Zahl
const myBoolean = true; // Boolean
```

Die einfachen Datentypen können in *komplexere* Strukturen organisiert werden, wie z. B. in ein *array* (Liste) oder ein *object* (Schlüssel-Wert-Paare).

```
/*
Arrays werden mit eckigen Klammern `[]` definiert, die Elemente
werden durch Kommas getrennt.
Ein Array kann Elemente unterschiedlichen Typs enthalten, aber
in der Praxis werden sie oft für Daten des gleichen Typs
verwendet.
*/
const myArray = [1, 2, 3, 4, 5]; // Ein Array von Zahlen
const myMixedArray = ["Hello", 42, true]; // nicht empfohlen

// Objekte werden mit geschwungenen Klammern `{}` definiert,
// die Schlüssel-Wert-Paare enthalten.
const myObject = { // Objekt
  name: "MapLibre",
  type: "Webmapping Library",
  openSource: true,
};
```

Funktionen und Methoden

Funktionen sind wiederverwendbare Codeblöcke, die eine bestimmte Aufgabe erfüllen. Sie können Parameter entgegennehmen und einen Wert zurückgeben. Funktionen können auf verschiedene Arten definiert werden, z. B. als Funktionsdeklaration oder als Funktionsausdruck.

```
// Funktionsdeklaration mit dem Schlüsselwort `function`
function sayHello(name) {
  return `Hallo, ${name}!`;
}

// Funktionsausdruck, bei dem eine Funktion einer Variablen
// zugewiesen wird
const sayHello = (name) => {
  return `Hallo, ${name}!`;
};

// Der Funktionsaufruf erfolgt immer gleich, unabhängig von der
// Art der Definition
console.log(sayHello("MapLibre")); // Ausgabe: Hallo, MapLibre!
```

Weiterführende Informationen

MapLibre-Bibliothek

- Dokumentation der *MapLibre GL JS* API:
 - API-Dokumentation: <https://maplibre.org/maplibre-gl-js/docs/API/>
 - Beispiele: <https://maplibre.org/maplibre-gl-js/docs/examples/>
- *MapLibre*-Style-Spezifikationen: <https://maplibre.org/maplibre-style-spec/>

Stile und Assets

- basemap.de: <https://basemap.de/produkte-und-dienste/web-vektor/>
- OpenFreeMap: https://openfreemap.org/quick_start/
- Terrain Tiles: <https://registry.opendata.aws/terrain-tiles/>
- MapLibre-Assets: <https://latitudemaps.github.io/basemap-assets/>

Vectortiles erstellen

Mit dem Tool [tippecanoe](#) können eigene Vectortiles erstellt werden. *tippecanoe* ist ein Kommandozeilen-Tool das auf Basis von z. B. ein (oder mehreren) GeoJSON-Dateien Vectortiles generieren kann. Dabei bietet das Tool verschiedene Optionen zur Anpassung der Tiles (z. B. Zoomstufen, Vereinfachung, etc.) an. Unter anderem ermöglicht es die Erstellung von [Protomaps Vectortiles](#) (*.pmtiles*), die ohne einen Server direkt im Browser verwendet werden können. Die Dokumentation zur Installation und Nutzung von [tippecanoe](#) finden Sie auf [github](https://github.com/felt/tippecanoe#usage): <https://github.com/felt/tippecanoe#usage>



Den finalen Code für die Demo Karte sowie das Handout und die Slides finden Sie auf GitHub: <https://github.com/jakoblistabarth/dkk-maplibre-workshop/>

NOTIZEN

[illegible]